

Build & Deployment Process Analysis

Phoenix Repository: Current Architecture, Workflows, and Optimization Opportunities — REVISED WITH GIT FLOW ALIGNMENT

September 2026 | Kalderos LLC

Contents

1. Executive Summary
2. Current Architecture Overview
3. CI Pipeline (Build Process)
4. Artifact Management
5. Environment Progression
6. Deployment Process
7. Release Management
8. Hotfix Process & Selective Deployments
9. Pain Points & Issues
10. Recommendations

1. Executive Summary

Phoenix currently uses **GitHub Actions for CI/CD** to build and deploy 14+ applications across multiple environments. The process is partially automated (Dev/Automated environments) but becomes increasingly manual further down the pipeline (QA, Preview, Prod). A custom hotfix process allows for selective artifact deployments when bugs are discovered in later environments.

Key Findings

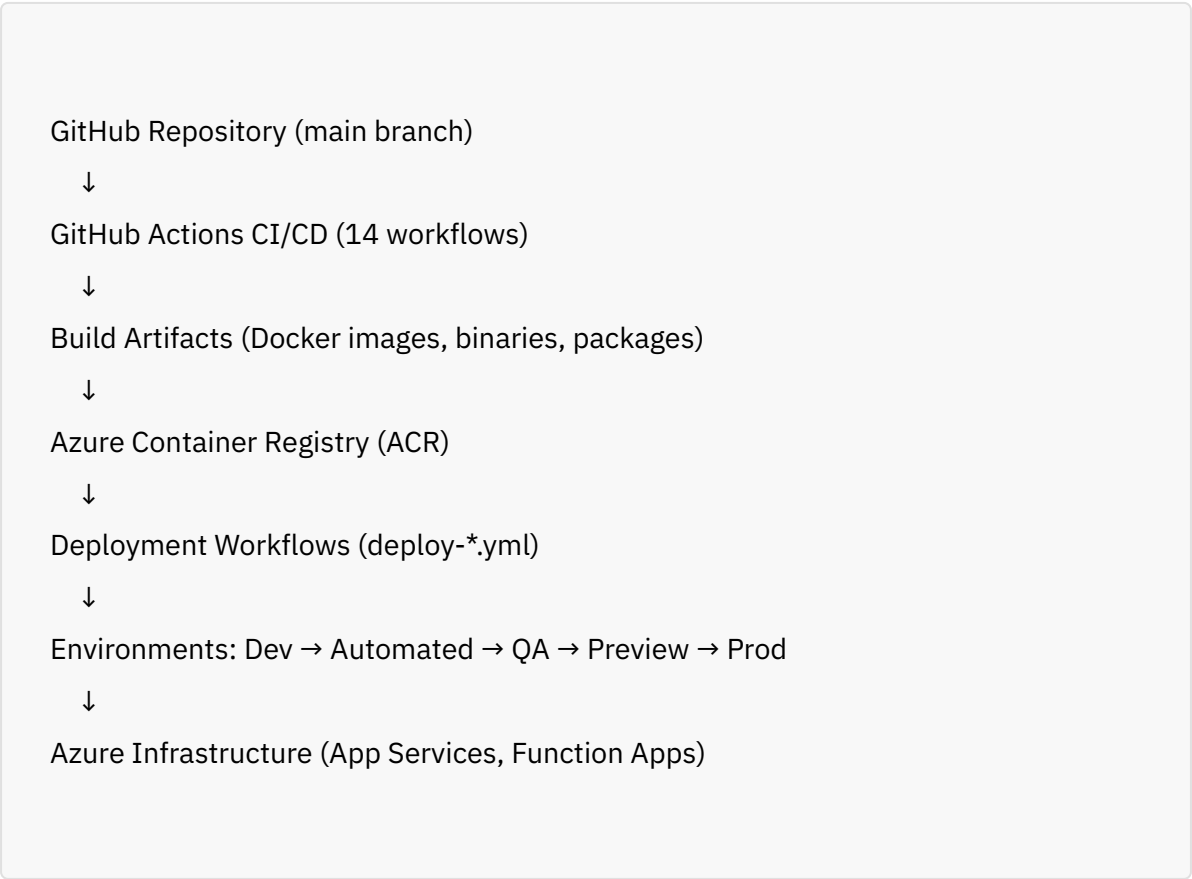
- **14+ applications** managed in a monorepo with independent build pipelines
- **Dual environment models:** Automatic deploys (Dev/Automated) vs. manual approval gates (QA/Preview/Prod)
- **Selective deployments:** Hotfixes can deploy individual apps, not entire monorepo
- **Kalderos manages Gilead environments:** Test → Stage progression (Stage = their production)
- **Manual orchestration:** No central dashboard showing deployment status across environments

Immediate Opportunities

- Automate artifact promotion (currently requires manual triggers)
- Improve visibility into which artifacts are in which environments via `kald-devops list_pipelines --json`
- Simplify hotfix process (currently complex branch/tag creation)
- Standardize client-specific deployments (Gilead logic is ad-hoc)

2. Current Architecture Overview

High-Level Flow



Component Breakdown

Component	Technology	Purpose
Source Control	GitHub (KalderosLLC/phoenix)	Code repository, release gate (main branch)
CI/CD Engine	GitHub Actions	Build, test, artifact creation
Artifact Registry	Azure Container Registry (ACR)	Stores Docker images for all 14 apps

Infrastructure as Code	Terraform (ARM templates)	Defines App Services, Function Apps, resources
Compute	Azure App Services, Function Apps	Runs applications in each environment
Orchestration	Manual workflow triggers	Deploy specific artifacts to specific environments

Applications & Build Artifacts

Category	Applications (14+)	Artifact Type
Azure Functions	Payment, EDI, Events, Notification, Geo, TPA, Kyriba, Enforcement, Client Transfer, Phoenix Core, Eligibilities	Docker image + Function App deployment
APIs	pay-api, request-api, phoenix-admin-api	Docker image + App Service deployment
Web UIs	pay-web, request-web, phoenix-admin-web	Compiled JavaScript + App Service deployment

3. CI Pipeline (Build Process)

Trigger Points

Trigger	When it Runs	Output
Pull Request	Developer opens PR to main	Unit tests, code coverage reports, no artifact publish
Push to main	Code merged to main branch	Full build, artifact created, tagged with commit SHA
Release tag	Semantic version tag created (v0.1.32)	Artifact re-tagged with version, promoted to "release" status
Hotfix tag	Hotfix branch tagged (v0.1.32.1)	New artifact built from hotfix commit, separate from main release

Build Process for Each Application

.NET Applications (APIs, Functions)

1. Checkout repository 2. Merge main into PR (for PR testing) 3. Restore NuGet packages 4. Run unit tests (dotnet test) 5. Build in Release mode (dotnet build -c Release) 6. Create Docker image from Dockerfile 7. Push Docker image to ACR with tag: - On main: azurecr.io/app-name:commit-sha - On release: azurecr.io/app-name:v0.1.32 - On hotfix: azurecr.io/app-name:v0.1.32.1

NPM Applications (Web UIs)

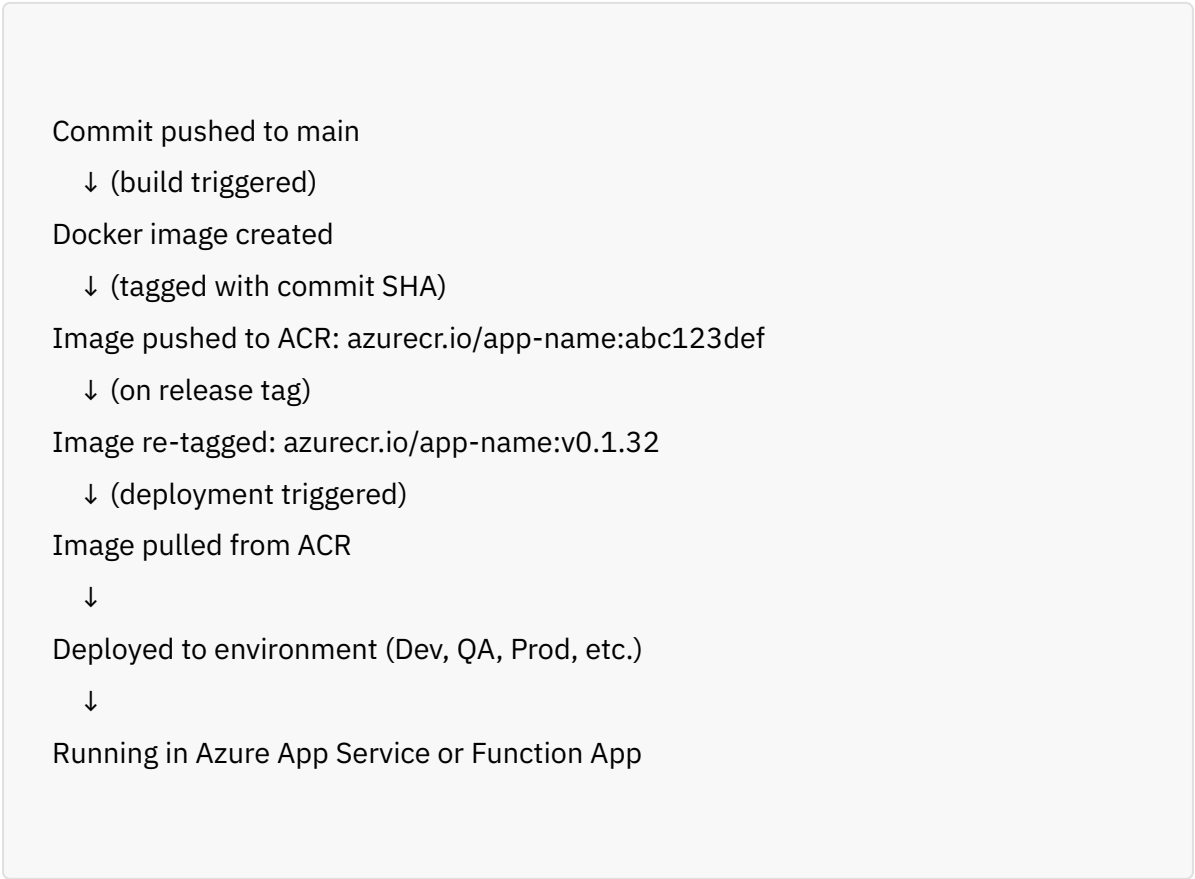
1. Checkout repository 2. Install dependencies (npm install) 3. Run tests (npm test) 4. Build for Release (npm run build) 5. Create Docker image containing compiled JavaScript 6. Push Docker image to ACR with same tagging scheme

Test Coverage Requirements

- **PR validation:** Unit tests must pass, code coverage must meet threshold
- **Main branch:** All unit tests run before artifact creation
- **Integration tests:** Separate workflow (phoenix-integration-tests-ci.yml) runs full system tests
- **No E2E in pipeline:** E2E tests run manually or in environments after deployment

4. Artifact Management

Artifact Lifecycle



Artifact Storage

Artifact Type	Storage Location	Retention Policy	Notes
Docker Images	Azure Container Registry (ACR)	All tags kept (commit SHA + version)	Pull-based deployment
Build Logs	GitHub Actions	90 days (GitHub default)	Contain test results, build output

Test Reports	GitHub Actions Artifacts	90 days (GitHub default)	Code coverage, test results
--------------	--------------------------	--------------------------	-----------------------------

Artifact Versioning Strategy

Current approach: Docker image tags follow these patterns:

- `app-name:abc123def` → Commit SHA (temporary, for PR testing)
- `app-name:v0.1.32` → Semantic version (release tag)
- `app-name:v0.1.32.1` → Patch version (hotfix tag)
- `app-name:latest` → Always points to latest main branch build

Issue: No explicit linkage between artifact version and deployment environment.
Unknown which version is currently in Prod.

5. Environment Progression

Standard Flow (Most Applications)

Environment	Deployment Trigger	Manual Approval	User Base	Typical Artifacts
Dev	Automatic on main push	No	Developers	Latest main (commit SHA)
Automated	Automatic on main push	No	Automated Testing	Latest main (commit SHA)
QA	Manual deploy workflow trigger	Yes (QA Lead)	QA Team	Release tag (v0.1.32)
Preview	Manual deploy workflow trigger	Yes (Manager)	Stakeholders	Release tag (v0.1.32)
Prod	Manual deploy workflow trigger	Yes (Director+)	End Users	Release tag (v0.1.32)

Gilead Client Flow (Kalderos Manages)

Exception: Gilead is a client whose environments are managed by Kalderos, using different progression than standard applications.

Environment	Purpose	Deployment
Test	Client testing environment	Manual trigger from Kalderos
Stage	Client production environment	Manual trigger from Kalderos

Environment Characteristics

Aspect	Dev/ Automated	QA/Preview	Prod
Data	Synthetic / Sandbox	Test data set	Live production data
Users	Internal only	Stakeholders, QA team	External, paying customers
Scale	Low (dev instances)	Medium (staging)	High (production scale)
Rollback Window	Immediate	Hours	Minutes
Downtime Impact	None (dev only)	Low (testing only)	High (revenue impact)

6. Deployment Process

Deployment Workflows

Phoenix uses manual deployment workflows triggered via GitHub Actions UI:

- `deploy-all.yml` → Deploy pay, request, admin APIs and Functions
- `deploy-geo.yml` → Deploy Geo function with staged promotion
- `deploy-eligibilities.yml` → Deploy eligibilities service
- `deploy-perftest.yml` → Deploy perftest infrastructure

Typical Deployment Workflow

1. Developer/Manager triggers `deploy-all.yml` workflow
2. Selects environment: dev, test, automated, stage, prod, or demo
3. Optionally specifies `run_id` (for selecting specific build)
4. Workflow executes:
 - a. Retrieves Docker image from ACR for that version
 - b. Runs Terraform to update infrastructure
 - c. Updates App Service / Function App with new image
 - d. Runs health checks / smoke tests
5. Deployment complete (or rolled back if tests fail)

Deployment Parameters

Parameter	Options	Purpose
environment	dev, test, automated, stage, prod, demo	Target environment for deployment

run_id	GitHub Actions build run ID (optional)	Specify which build artifacts to deploy
--------	---	--

Selective Deployment (Key Feature)

Important: Deployments are **per-application, not per-entire-repo**.

The deploy-all.yml workflow independently deploys:

- pay-api (if changed)
- pay-web (if changed)
- request-api (if changed)
- request-web (if changed)
- phoenix-admin-api (if changed)
- phoenix-admin-web (if changed)
- Payments Function (if changed)
- EDI Function (if changed)
- + all other functions

Each can be deployed independently, allowing teams to fix bugs in one app without redeploying all 14 applications.

7. Release Management

Release Process (Git Flow)

Step	Actor	Action	Artifact Change
1. End of Sprint	Team Lead	Confirm all developers "done" with features	None
2. Create Release Branch	Release Manager	Create release/0.1.32 from main	None
3. Deploy to Preview	QA Lead	Trigger deploy from release/* branch	Build versioned artifacts
4. QA Testing	QA Team	Test in Preview; find bugs	None
5. Release Bugfixes	Dev Team	Fix bugs directly in release/* branch	None (same tag)
6. Re-test & Approve	QA Team	Re-test in Preview	None
7. Merge & Tag	Release Manager	Merge release/* to main, tag v0.1.32	Release ready
8. Deploy to Prod	Director	Deploy v0.1.32 to production	Live

Key Advantage: Bugs Fixed Before Production

With Git Flow release branches: All bugs are caught and fixed in Preview BEFORE tagging main. This means:

- No patch versions needed for bugs found in testing
- v0.1.32 goes to Prod fully tested
- Hotfixes are rare (only for real production escapes)

8. Hotfix Process & Selective Deployments

Release Bugfix Workflow (Most Common Pattern)

When bugs are discovered in Preview during pre-release validation:

1. QA/Tester discovers bug in Preview environment
2. Fix is committed directly to release/0.1.32 branch
3. Dev makes fix in release/0.1.32 (no new branch needed)
4. Re-deploy to Preview with fixed artifact
5. Testing resumes with mixed app versions
6. Once all tests pass, release/* branch is ready
7. Merge release/0.1.32 to main (--no-ff)
8. Tag main: git tag v0.1.32 (no patch needed)
9. Build triggers, creates release artifacts
10. Deploy to Prod from v0.1.32 tag

Hotfix Workflow (Production Bug After Release)

If a critical bug slips through and reaches production:

1. Production issue discovered after v0.1.32 released
2. Create hotfix/* branch from main
3. Dev fixes bug in hotfix/critical-fix branch
4. Deploy to Preview for validation with production data
5. Merge hotfix/* to main (--no-ff)
6. Tag: git tag v0.1.32.1 (patch version)
7. Build triggers, creates patch artifacts
8. Merge hotfix/* to develop (--no-ff)

9. Deploy v0.1.32.1 patch to Prod

Key: Most Fixes Are Release Bugfixes (Not Hotfixes)

Important Pattern: The most common scenario is bugs found in Preview during release/* branch testing. These are **release bugfixes**, not hotfixes. They:

- Are fixed directly in release/* branch
- Get tested in Preview with mixed app versions
- Get merged to main with no patch version needed
- Prevent hotfixes from being necessary

True hotfixes (from main) only happen when bugs slip through to production.

Selective Artifact Deployment in Hotfixes

Key Feature: Hotfix deployments deploy **ONLY** the artifacts that were fixed, not the entire 14-app suite.

Example:

- Bug found: payment-functions has a critical issue in Preview
- Team creates hotfix from v0.1.32
- Fixes bug in payment-functions only
- Tags as v0.1.32.1
- Tests in Preview with fixed payment-functions + stable other apps (mixed versions)
- Deploys **ONLY** payment-functions:v0.1.32.1 to Prod

- All other apps remain on v0.1.32

Artifact Versioning Across Environments

Application	Preview	Prod	Notes
payment-functions	v0.1.32.1	v0.1.32.1	Hotfix deployed
pay-api	v0.1.32	v0.1.32	No changes, same across all
edi-functions	v0.1.32	v0.1.32	No changes, same across all
pay-web	v0.1.32	v0.1.32	No changes, same across all

Hotfix Challenges

1. Manual Artifact Tracking: No centralized dashboard showing which app versions are in which environments. Spreadsheet-based tracking is error-prone.

2. Complex Process: Create-hotfix-branches.yml adds ceremony. Could be simplified with automation.

3. Version Proliferation: Many patch versions (v0.1.32.1, v0.1.32.2, v0.1.32.3) can accumulate.

4. Deployment Risk: Manual selection of which artifacts to deploy risks deploying wrong version by mistake.

9. Pain Points & Issues

Issue 1: Manual Orchestration

Problem: Each deployment requires manual workflow trigger via GitHub UI. No automation between environments.

- **Impact:** Slow deployment, prone to human error, difficult to coordinate across teams
- **Example:** To promote from QA to Preview to Prod requires 3 separate manual triggers
- **Frequency:** Happens weekly per release, 3-5 times per release (including hotfixes)

Issue 2: Artifact Visibility

Problem: No centralized view of which artifact versions are in which environments.

- **Current approach:** Check ACR tags manually, or rely on tribal knowledge
- **Solution available:** `kald-devops list_pipelines --json` provides this visibility — needs integration
- **Deployment risk:** Accidental re-deployment or wrong version

Issue 3: Hotfix Complexity

Problem: `create-hotfix-branches.yml` adds 5-10 minutes of ceremony to emergency fixes.

- **Steps:** Trigger workflow → confirm version inputs → wait for branches → create PR → merge → tag → build → deploy

- **For critical bugs:** Every minute counts. This process is too slow.
- **Alternative:** Could directly create hotfix commit + tag + deploy in 2-3 minutes

Issue 4: Environment-Specific Deployments

Problem: Gilead client uses Test → Stage flow, but no automation for different customer deployments.

- **Current:** Ad-hoc manual steps, easy to miss a deployment
- **Scalability:** If more clients need custom flows, process becomes unmanageable

Issue 5: No Artifact Promotion Policy

Problem: No formal policy for when/how artifacts move between environments.

- **Unclear gates:** What triggers QA → Preview promotion? No automated policy.
- **Compliance risk:** Hard to audit deployment approvals and handoffs

Issue 6: Testing After Release Tag

Problem: Release version is tagged (v0.1.32) BEFORE any QA testing. Leads to patch versions for bugs found later.

- **Better approach:** Test on release/* branch before tagging main (Git Flow)
- **This improves the release process significantly**

Summary of Pain Points

Issue	Frequency	Severity	Impact
		Medium	Slow, error-prone

Manual orchestration	Every deployment		
Artifact visibility	Ongoing	High	Deployment risks, unclear state
Hotfix complexity	Weekly	High	Slow emergency response
Client-specific flows	Per-client	Medium	Manual, unscalable
No promotion policy	Ongoing	Medium	Compliance, audit trail
Testing after tag	Every release	High	Leads to patch versions

10. Recommendations

Short-Term (1-2 Weeks): Quick Wins

Recommendation 1: Wire Up kald-devops for Artifact Visibility

Use `kald-devops list_pipelines --json` to show deployment state:

- All 14 applications
- Version currently in each environment (Dev, QA, Preview, Prod, Gilead Test/Stage)
- Last deployment timestamp
- Link to ACR image details

Implementation: Call via scheduled workflow, store JSON output, display on dashboard

Time: 3-5 hours

Value: Eliminates artifact guessing, improves safety

Recommendation 2: Simplify Hotfix Tagging

Replace `create-hotfix-branches.yml` with simpler approach:

- Developer directly commits fix to hotfix/* branch
- Creates git tag directly: `git tag v0.1.32.1`
- Push tag triggers build automatically
- Manual deploy (same as now)

Time saved: 5-10 minutes per hotfix

Simpler: One-step instead of three

Medium-Term (2-4 Weeks): Core Improvements

Recommendation 3: Implement Git Flow with Release Branches

Adopt formal Git Flow branching:

- 1 Create and protect `develop` branch
- 2 All features merge to develop (not main)
- 3 At sprint end, create `release/0.2.0` from develop
- 4 Test release/* branch in Preview BEFORE tagging main
- 5 Tag main only after release/* validation
- 6 Hotfixes created from main, merged back to both main and develop

Timeline: 2-3 weeks

Impact: Eliminates patch versions, cleaner history, fewer tags

Long-Term (1-3 Months): Strategic Improvements

Recommendation 4: Automated Environment Promotion

Automate artifact flow between environments:

- 1 Define promotion gates (Preview passes → auto-promote to Prod on approval)
- 2 Add approval checkpoints (Preview → Prod requires director approval)
- 3 Implement promotion workflow that triggers deploys automatically
- 4 Keep audit trail of who approved each promotion

Timeline: 2-3 weeks

Impact: Faster releases, better auditability, reduced manual steps

Recommendation 5: Standardize Client-Specific Flows

Create configuration for different deployment patterns:

- Standard: Dev → Automated → QA → Preview → Prod
- Gilead: Dev → Automated → Test → Stage
- Configuration in repo (deployment-config.yaml)
- Deploy workflows read config to determine environment targets

Timeline: 1 week

Impact: Scales to multiple clients, reduces manual configuration

Implementation Priority Matrix

Recommendation	Effort	Impact	Priority
Wire up kald-devops	Low (3-5 hrs)	High (visibility)	Do First
Simplify hotfix tagging	Low (2 hrs)	High (speed)	Do First
Implement Git Flow	Medium (2-3 wks)	High (clarity)	Next
Automated promotion	Medium (2-3 wks)	High (efficiency)	Next
Standardize client flows	Medium (1 wk)	Medium (scalability)	Next

Risks of Not Addressing These Issues

- **Deployment failures:** Manual orchestration is error-prone; accidental re-deployments or wrong versions
- **Slower incident response:** Complex hotfix process delays critical fixes
- **Compliance gaps:** No audit trail of deployment approvals
- **Scaling problems:** Adding more clients makes manual flows unsustainable
- **Version confusion:** Too many patch versions, unclear which is actually in production

Document prepared for Kalderos LLC | September 2026

Updated: Kalderos manages Gilead environments (Stage = production), hotfixes primarily from Preview, Git Flow alignment, kald-devops integration